



# Introduction to Computer Networks (SWE3022)

Jaehoon (Paul) Jeong

Department of Computer Science and Engineering

Sungkyunkwan University

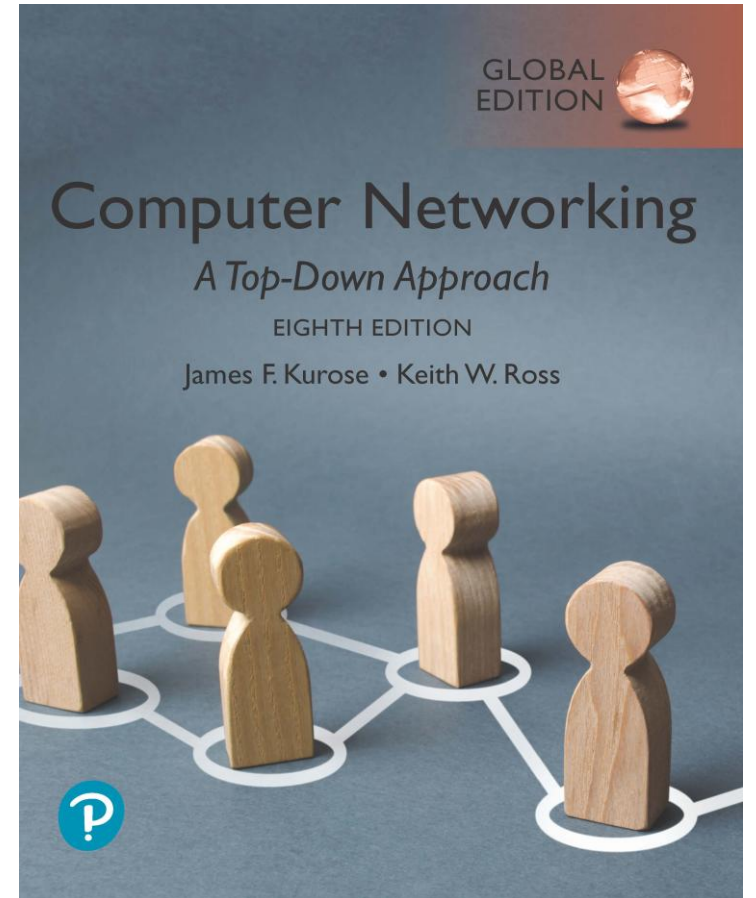
Email: [pauljeong@skku.edu](mailto:pauljeong@skku.edu)

Note: The slides are adapted from the slides of Pearson Education Ltd.

# Chapter 4.

## Network Layer - Data Plane:

### Part 1



## *Computer Networking: A Top-Down Approach*

8<sup>th</sup> Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

# Network layer: our goals

- understand principles behind network layer services, focusing on data plane:
  - network layer service models
  - forwarding versus routing
  - how a router works
  - addressing
  - generalized forwarding
  - Internet architecture
- instantiation, implementation in the Internet
  - IP protocol
  - NAT, middleboxes

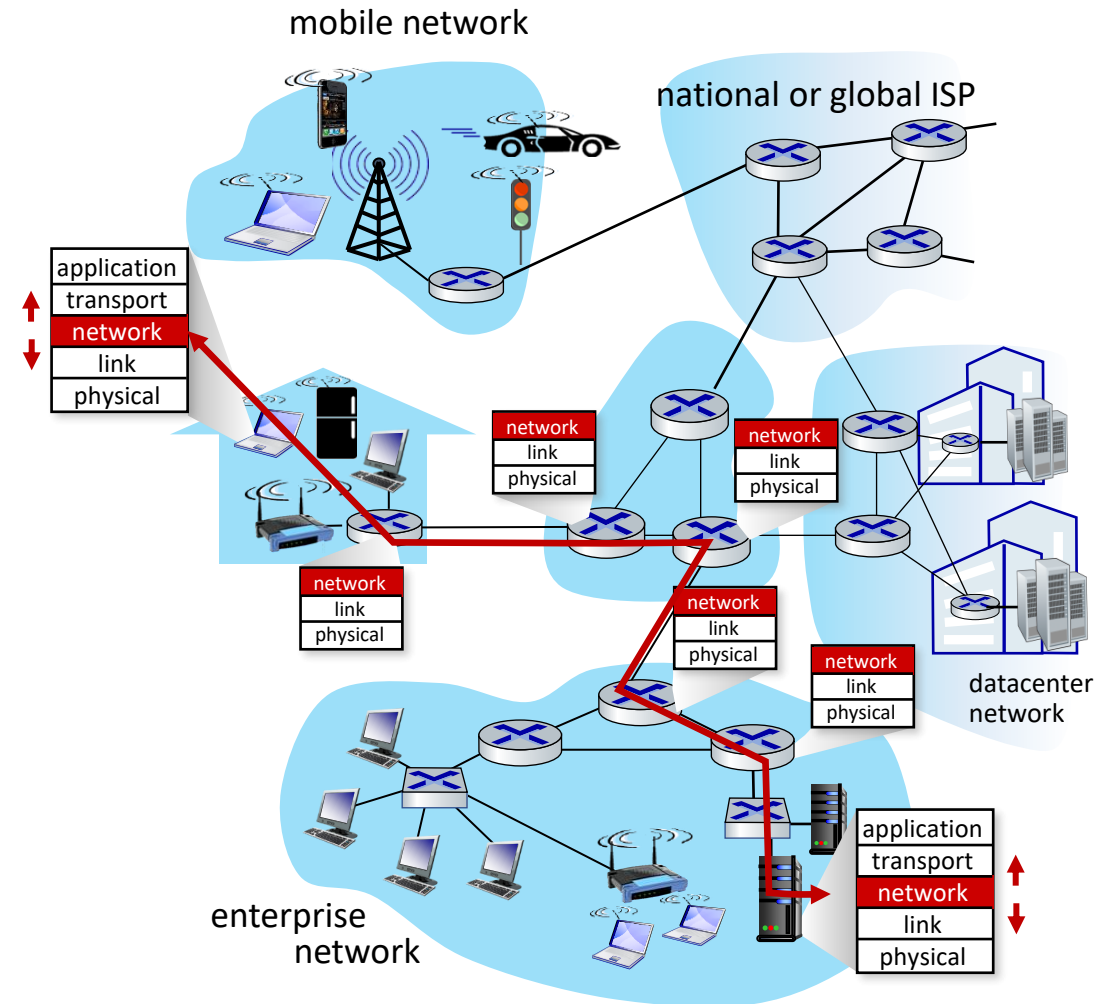
# Network layer: “data plane” roadmap

- Network layer: overview
  - data plane
  - control plane
- What’s inside a router
  - input ports, switching, output ports
  - buffer management, scheduling
- IP: the Internet Protocol
  - datagram format
  - addressing
  - network address translation
  - IPv6
- Generalized Forwarding, SDN
  - Match+action
  - OpenFlow: match+action in action
- Middleboxes



# Network-layer services and protocols

- transport segment from sending to receiving host
  - **sender:** encapsulates segments into datagrams, passes to link layer
  - **receiver:** delivers segments to transport layer protocol
- network layer protocols in *every Internet device*: hosts, routers
- **routers:**
  - examines header fields in all IP datagrams passing through it
  - moves datagrams from input ports to output ports to transfer datagrams along end-end path



# Two key network-layer functions

## network-layer functions:

- **forwarding**: move packets from a router's input link to appropriate router output link
- **routing**: determine route taken by packets from source to destination
  - *routing algorithms*

## analogy: taking a trip

- **forwarding**: process of getting through single interchange
- **routing**: process of planning trip from source to destination



forwarding



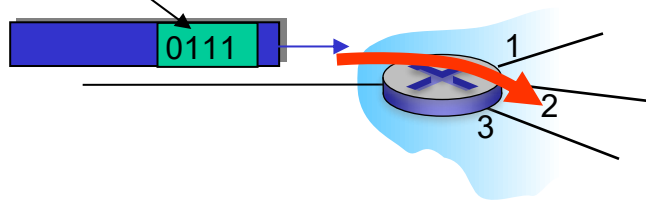
routing

# Network layer: data plane, control plane

## Data plane:

- *local*, per-router function
- determines how datagram arriving on router input port is forwarded to router output port

values in arriving  
packet header



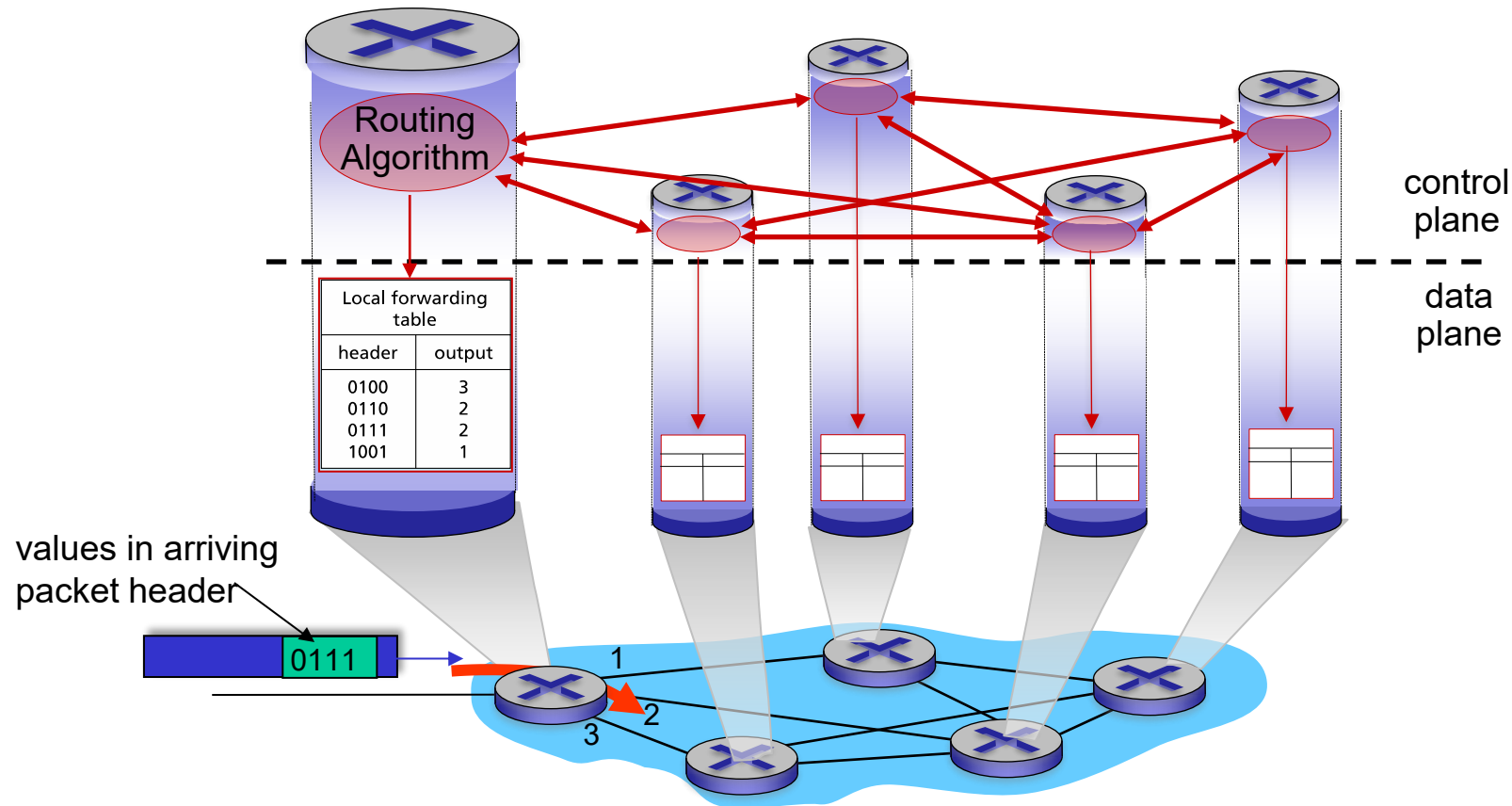
## Control plane

- *network-wide* logic
- determines how an IP datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
  - *traditional routing algorithms*: implemented in routers
  - *Software-Defined Networking (SDN)*: implemented in (remote) servers



# Per-router control plane

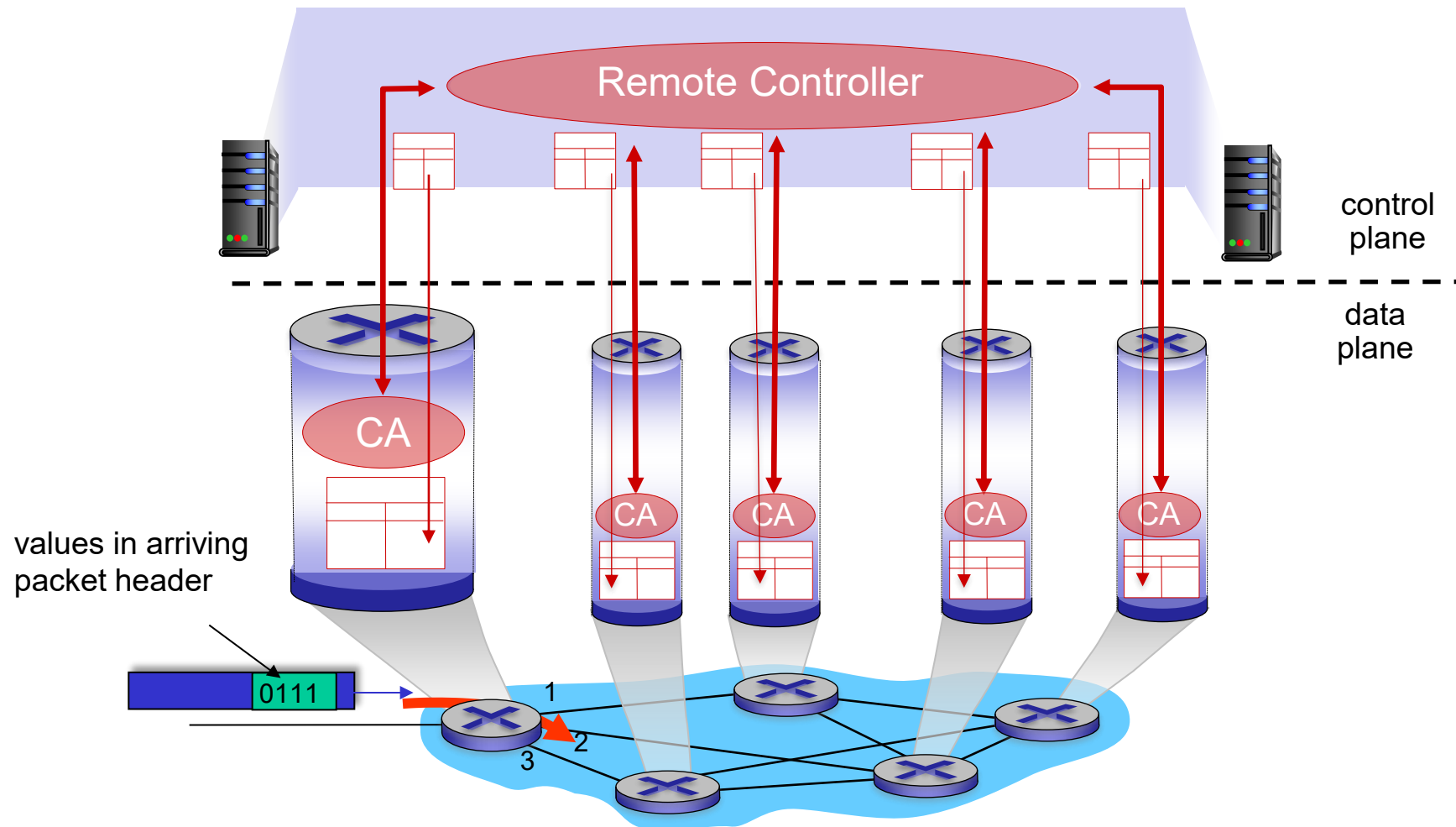
Individual routing algorithm components *in each and every router* interact in the control plane





# Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers



# Network service model

*Q:* What *service model* for “channel” transporting datagrams from sender to receiver?

example services for  
*individual* datagrams:

- guaranteed delivery
- guaranteed delivery with less than 40 msec delay

example services for a *flow* of  
datagrams:

- in-order datagram delivery
- guaranteed minimum bandwidth to flow
- restrictions on changes in inter-packet spacing

# Network-layer service model

| Network Architecture | Service Model | Quality of Service (QoS) Guarantees ? |      |       |        |
|----------------------|---------------|---------------------------------------|------|-------|--------|
|                      |               | Bandwidth                             | Loss | Order | Timing |
| Internet             | best effort   | none                                  | no   | no    | no     |

Internet “best effort” service model

*No* guarantees on:

- i. successful datagram delivery to destination
- ii. timing or order of delivery
- iii. bandwidth available to end-end flow

# Network-layer service model

| Network Architecture | Service Model                 | Quality of Service (QoS) Guarantees ? |          |          |        |
|----------------------|-------------------------------|---------------------------------------|----------|----------|--------|
|                      |                               | Bandwidth                             | Loss     | Order    | Timing |
| Internet             | best effort                   | none                                  | no       | no       | no     |
| ATM                  | Constant Bit Rate             | Constant rate                         | yes      | yes      | yes    |
| ATM                  | Available Bit Rate            | Guaranteed min                        | no       | yes      | no     |
| Internet             | Intserv Guaranteed (RFC 1633) | yes                                   | yes      | yes      | yes    |
| Internet             | Diffserv (RFC 2475)           | possible                              | possibly | possibly | no     |

[Reference for ATM] Broadband ISDN and Asynchronous Transfer Mode (ATM): <https://ieeexplore.ieee.org/abstract/document/35508>

# Reflections on best-effort service:

- **simplicity of mechanism** has allowed Internet to be widely deployed adopted
- sufficient **provisioning of bandwidth** allows performance of real-time applications (e.g., interactive voice, video) to be “good enough” for “most of the time”
- **replicated, application-layer distributed services** (datacenters, content distribution networks) connecting close to clients’ networks, allow services to be provided from multiple locations
- congestion control of “elastic” services helps

*It's hard to argue with success of best-effort service model*

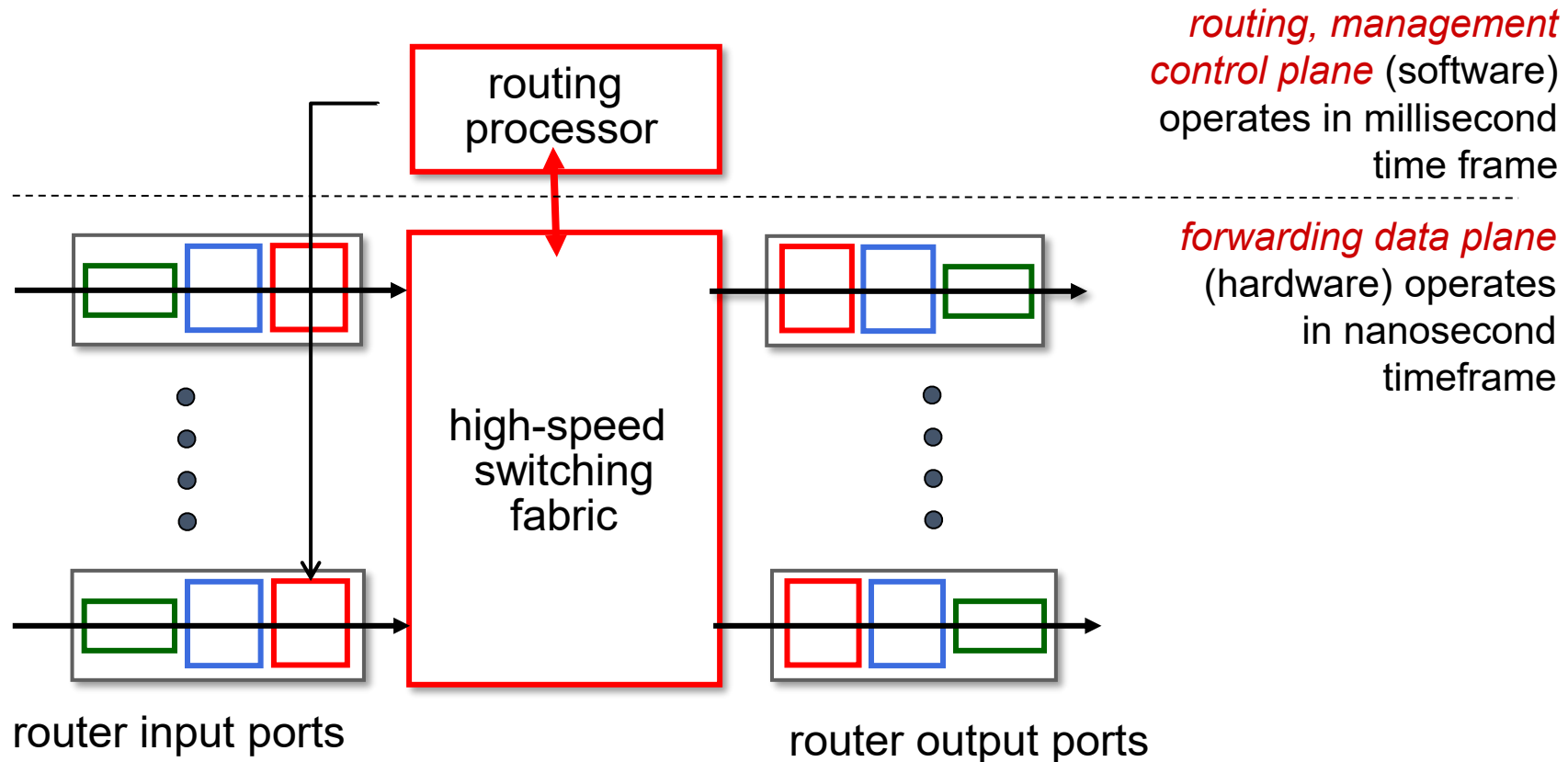
# Network layer: “data plane” roadmap

- Network layer: overview
  - data plane
  - control plane
- What’s inside a router
  - input ports, switching, output ports
  - buffer management, scheduling
- IP: the Internet Protocol
  - datagram format
  - addressing
  - network address translation
  - IPv6
- Generalized Forwarding, SDN
  - Match+action
  - OpenFlow: match+action in action
- Middleboxes



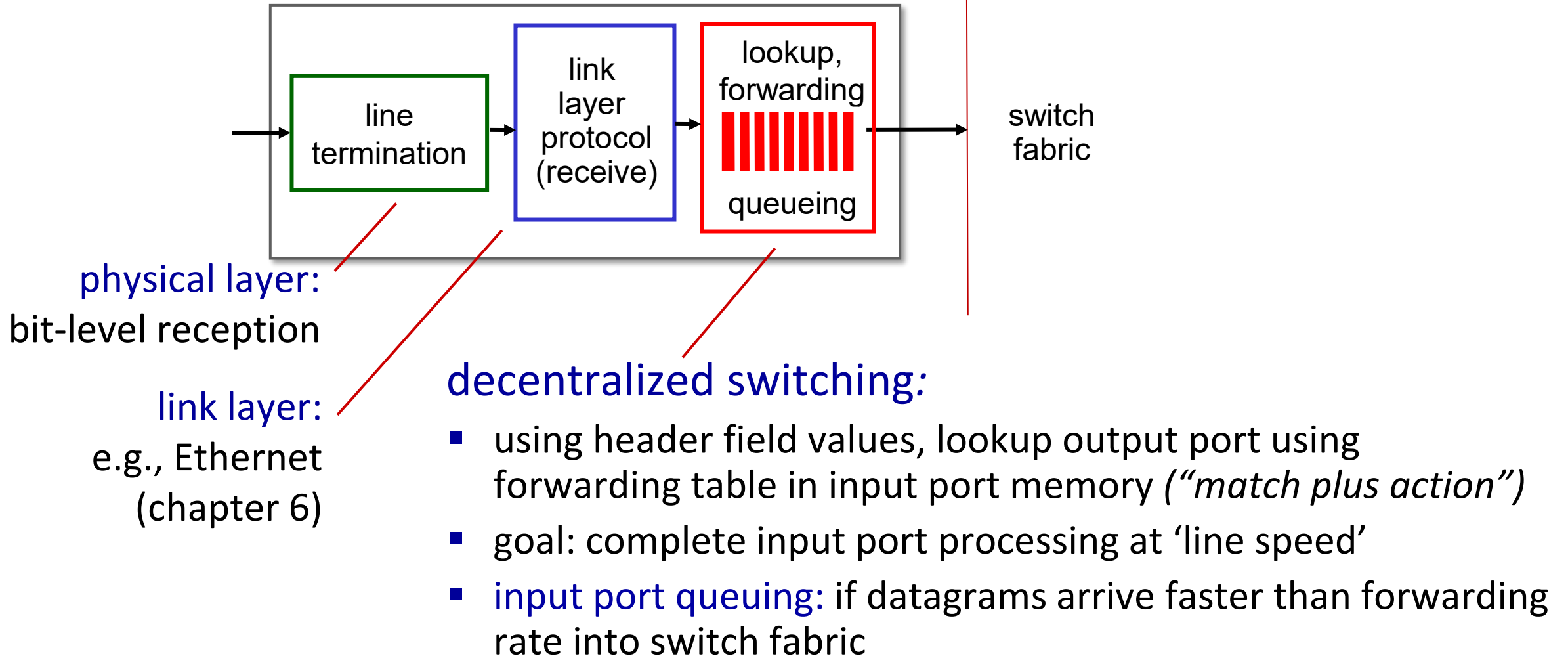
# Router architecture overview

high-level view of generic router architecture:

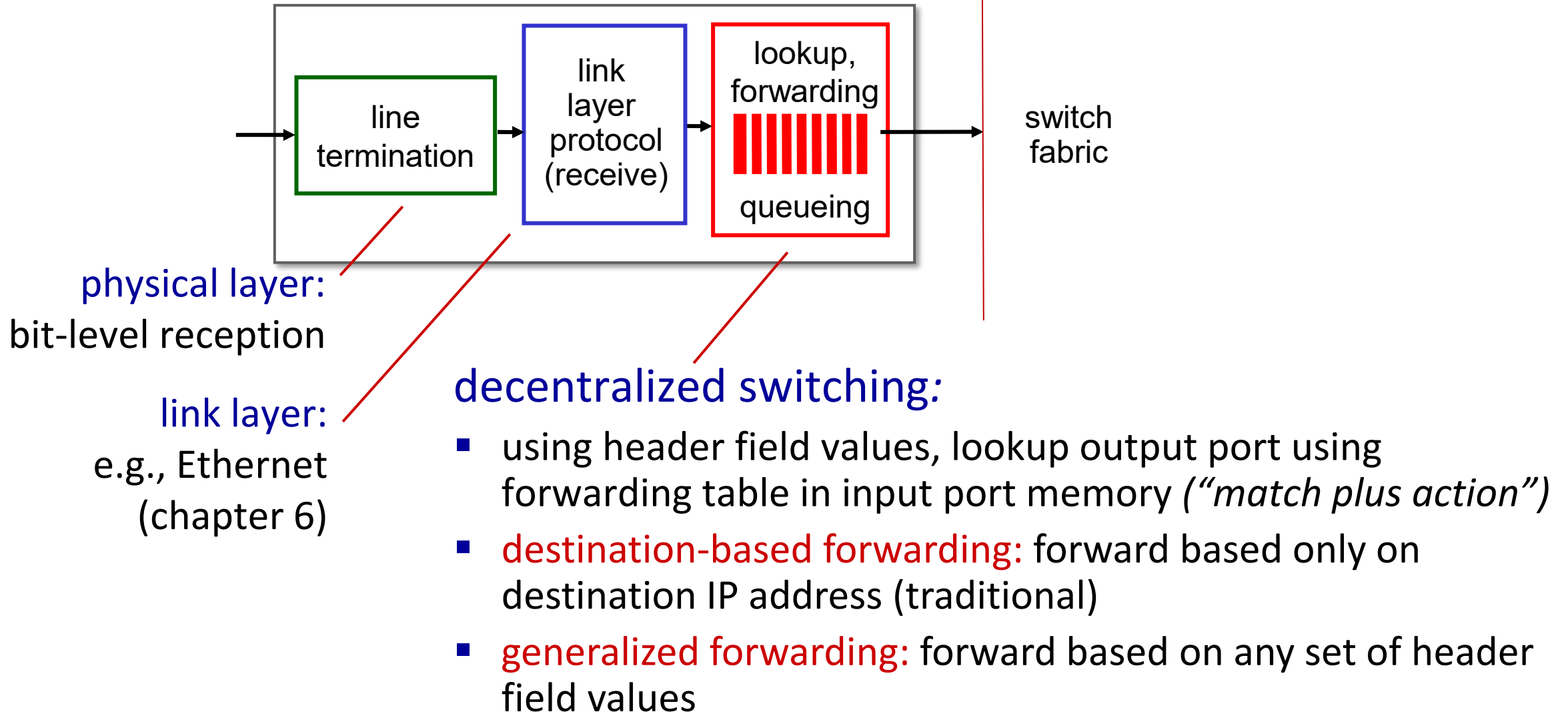




# Input port functions



# Input port functions



# Destination-based forwarding

| <i>forwarding table</i>                                                               |                |
|---------------------------------------------------------------------------------------|----------------|
| Destination Address Range                                                             | Link Interface |
| 11001000 00010111 00010000 00000000<br>through<br>11001000 00010111 00010000 00000100 | n<br><br>3     |
| 11001000 00010111 00010000 00000111                                                   |                |
| 11001000 00010111 00011000 11111111                                                   |                |
| 11001000 00010111 00011001 00000000<br>through<br>11001000 00010111 00011111 11111111 | 2              |
| otherwise                                                                             | 3              |

*Q:* but what happens if ranges don't divide up so nicely?

# Destination-based forwarding

| <i>forwarding table</i>                                                               |                |
|---------------------------------------------------------------------------------------|----------------|
| Destination Address Range                                                             | Link Interface |
| 11001000 00010111 00010000 00000000<br>through<br>11001000 00010111 00010111 11111111 | 0              |
| 11001000 00010111 00010000 00000100<br>through<br>11001000 00010111 00010000 00000111 | 3              |
| 11001000 00010111 00011000 00000000<br>through<br>11001000 00010111 00011000 11111111 | 1              |
| 11001000 00010111 00011001 00000000<br>through<br>11001000 00010111 00011111 11111111 | 2              |
| otherwise                                                                             | 3              |

# Longest prefix matching

## longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

| Destination Address Range                 | Link interface |
|-------------------------------------------|----------------|
| 11001000    00010111    00010***    ***** | 0              |
| 11001000    00010111    00011000    ***** | 1              |
| 11001000    00010111    00011***    ***** | 2              |
| otherwise                                 | 3              |

examples:

11001000    00010111    00010110    10100001    which interface?

11001000    00010111    00011000    10101010    which interface?

# Longest prefix matching

## longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

| Destination Address Range        | Link interface |
|----------------------------------|----------------|
| 11001000 00010111 00010*** ***** | 0              |
| 11001000 00010111 00011000 ***** | 1              |
| 11001000 match! 1 00011*** ***** | 2              |
| otherwise                        | 3              |

examples:

|          |          |          |          |                  |
|----------|----------|----------|----------|------------------|
| 11001000 | 00010111 | 00010110 | 10100001 | which interface? |
| 11001000 | 00010111 | 00011000 | 10101010 | which interface? |

# Longest prefix matching

## longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

| Destination Address Range        | Link interface |
|----------------------------------|----------------|
| 11001000 00010111 00010*** ***** | 0              |
| 11001000 00010111 00011000 ***** | 1              |
| 11001000 00010111 00011*** ***** | 2              |
| otherwise                        | 3              |

match!

examples:

|                                     |                  |
|-------------------------------------|------------------|
| 11001000 00010111 00010110 10100001 | which interface? |
| 11001000 00010111 00011000 10101010 | which interface? |

# Longest prefix matching

## longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

| Destination Address Range        | Link interface |
|----------------------------------|----------------|
| 11001000 00010111 00010*** ***** | 0              |
| 11001000 00010111 00011000 ***** | 1              |
| 11001000 00010111 00011*** ***** | 2              |
| otherwise                        | 3              |

match!

examples:

|                                     |                  |
|-------------------------------------|------------------|
| 11001000 00010111 00010110 10100001 | which interface? |
| 11001000 00010111 00011000 10101010 | which interface? |

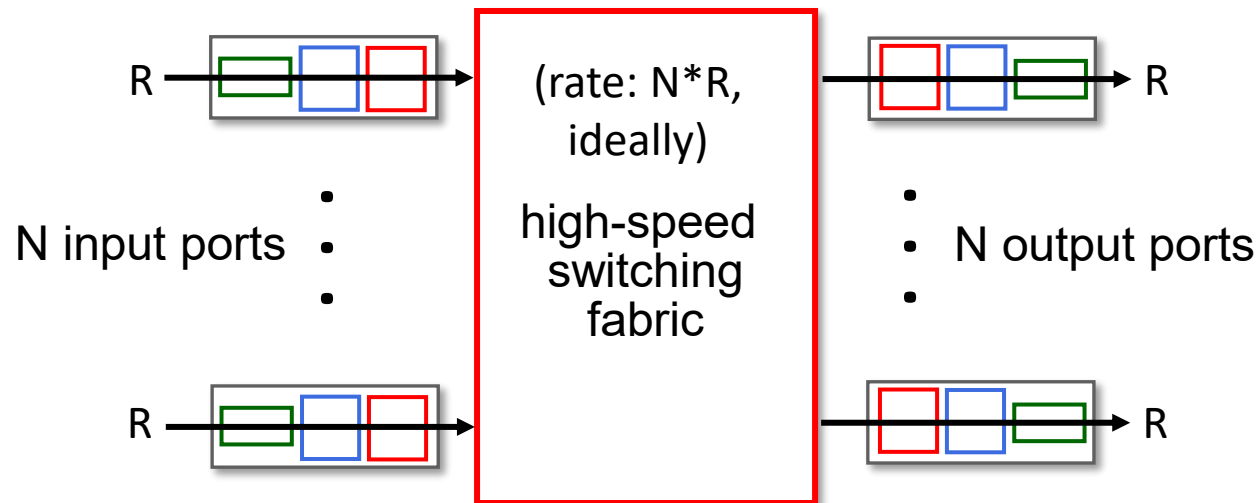


# Longest prefix matching

- we'll see *why* longest prefix matching is used shortly, when we study addressing
- longest prefix matching: often performed using ternary content addressable memories (TCAMs)
  - *content addressable*: present address to TCAM: retrieve address in one clock cycle, regardless of table size
  - Cisco Catalyst: ~1M routing table entries in TCAM

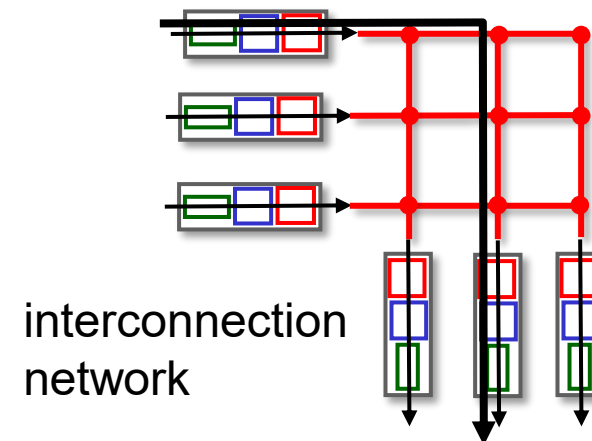
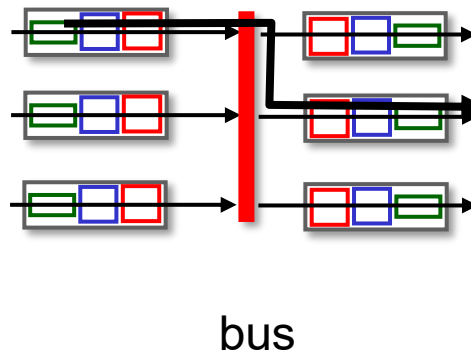
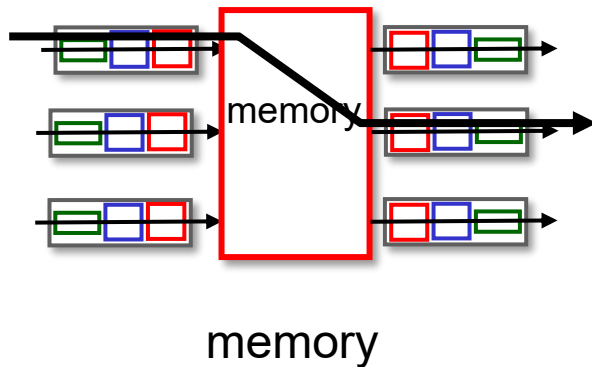
# Switching fabrics

- transfer packet from input link to appropriate output link
- **switching rate**: rate at which packets can be transfer from inputs to outputs
  - often measured as multiple of input/output line rate
  - N inputs: switching rate N times line rate desirable:  $N \cdot R$  for link rate R



# Switching fabrics

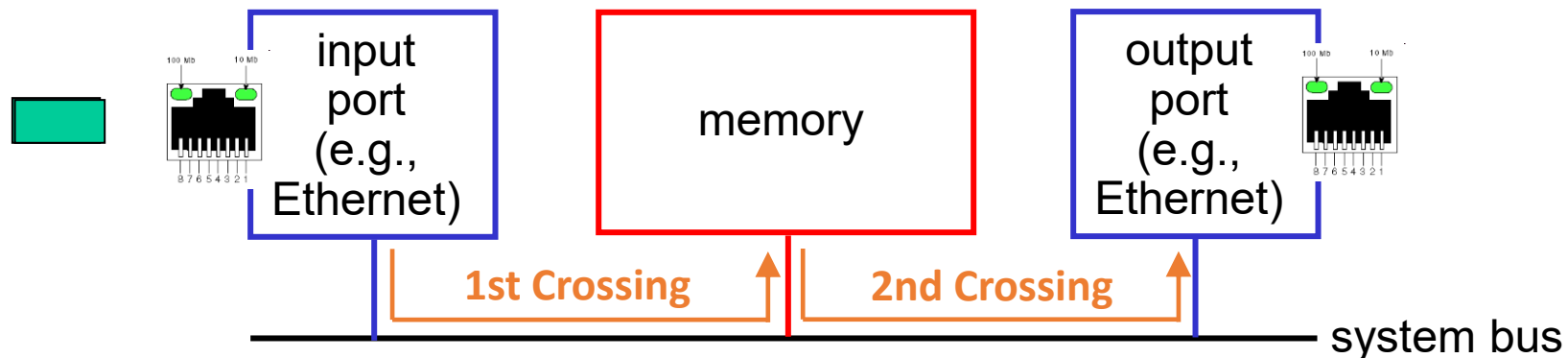
- transfer packet from input link to appropriate output link
- **switching rate**: rate at which packets can be transfer from inputs to outputs
  - often measured as multiple of input/output line rate
  - N inputs: switching rate N times line rate desirable
- three major types of switching fabrics:



# Switching via memory

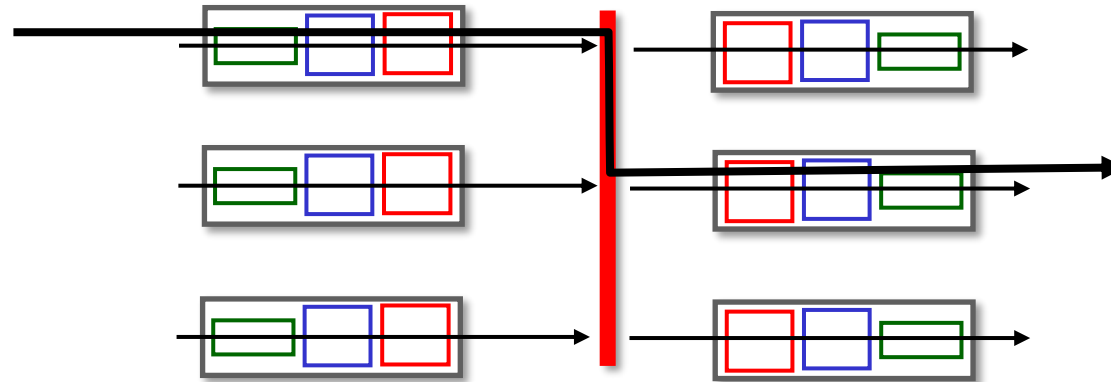
## first generation routers:

- Traditional computers with switching under direct control of CPU
- First of all, packet should be copied to system's memory for CPU.
- Speed limited by memory bandwidth (2 bus crossings per datagram)
  - Crossing from input port to memory
  - Crossing from memory to output port



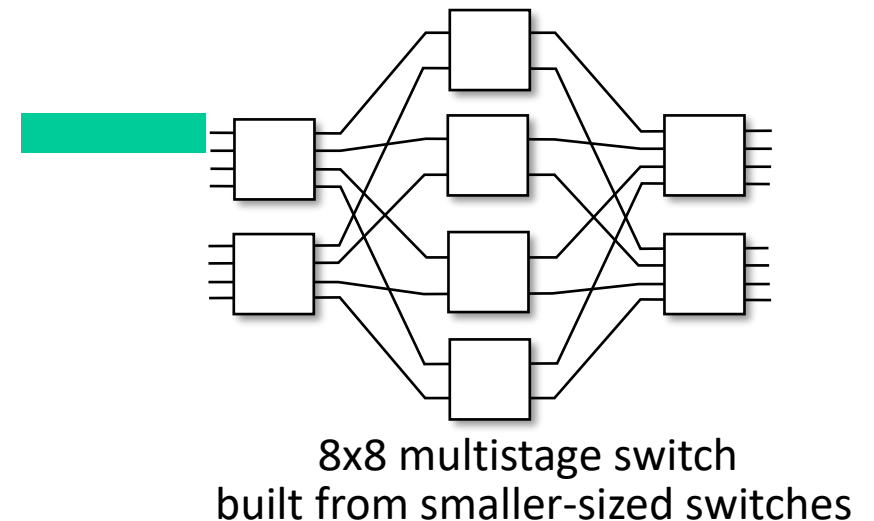
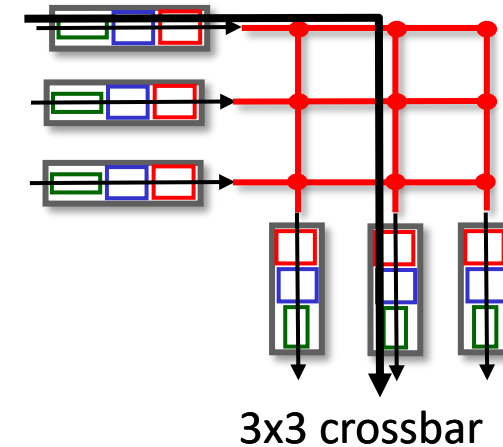
# Switching via a bus

- datagram from input port memory to output port memory via a shared bus
- *bus contention*: switching speed limited by bus bandwidth
- 32 Gbps bus, Cisco 5600: sufficient speed for access routers



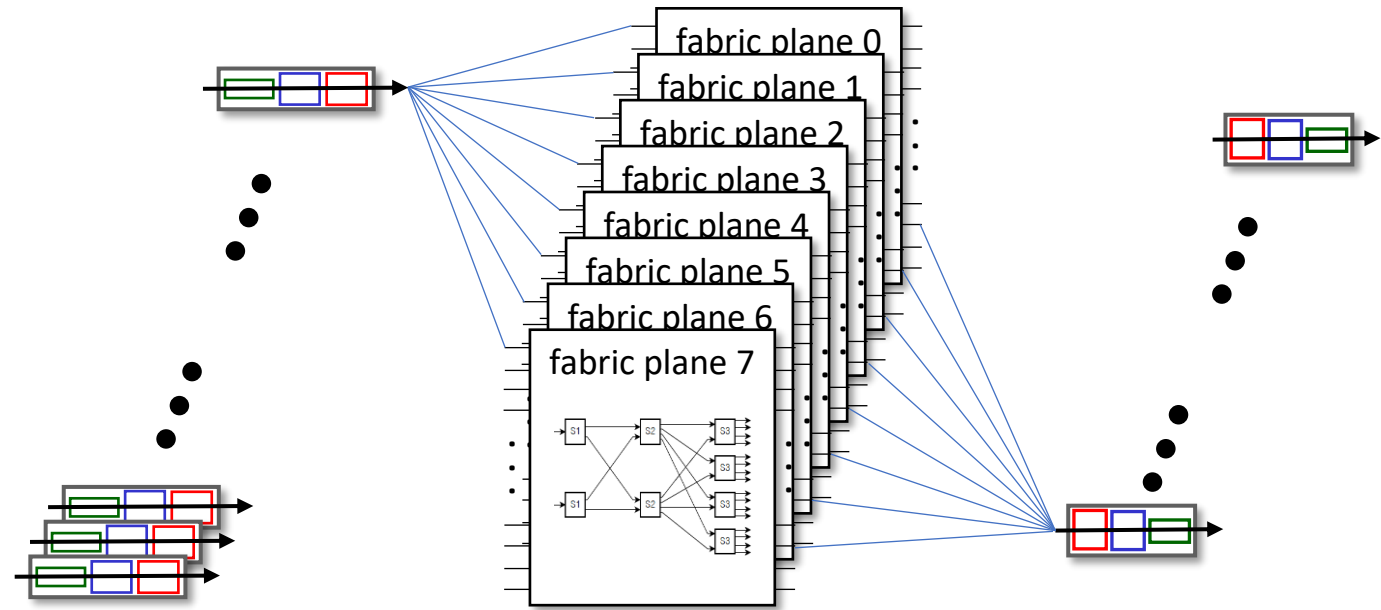
# Switching via interconnection network

- **Crossbar**, Clos networks, other interconnection nets initially developed to connect processors in multiprocessor
- **multistage switch**:  $n \times n$  switch from multiple stages of smaller switches
- **exploiting parallelism**:
  - fragment datagram into fixed length cells on entry
  - switch cells through the fabric, reassemble datagram at exit



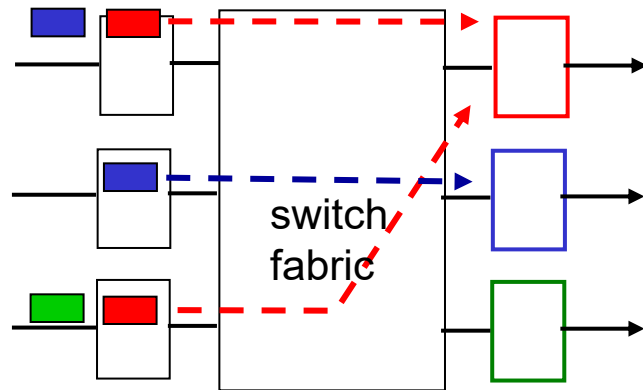
# Switching via interconnection network

- scaling, using multiple switching “planes” in parallel:
  - speedup, scaleup via parallelism
- Cisco CRS router:
  - basic unit: 8 switching planes
  - each plane: 3-stage interconnection network
  - up to 100's Tbps switching capacity

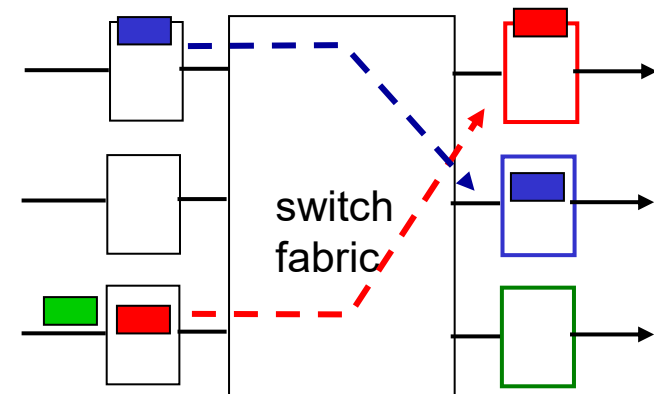


# Input port queuing

- If switch fabric slower than input ports combined -> queueing may occur at input queues
  - queueing delay and loss due to input buffer overflow!
- **Head-of-the-Line (HOL) blocking:** queued datagram at front of queue prevents others in queue from moving forward



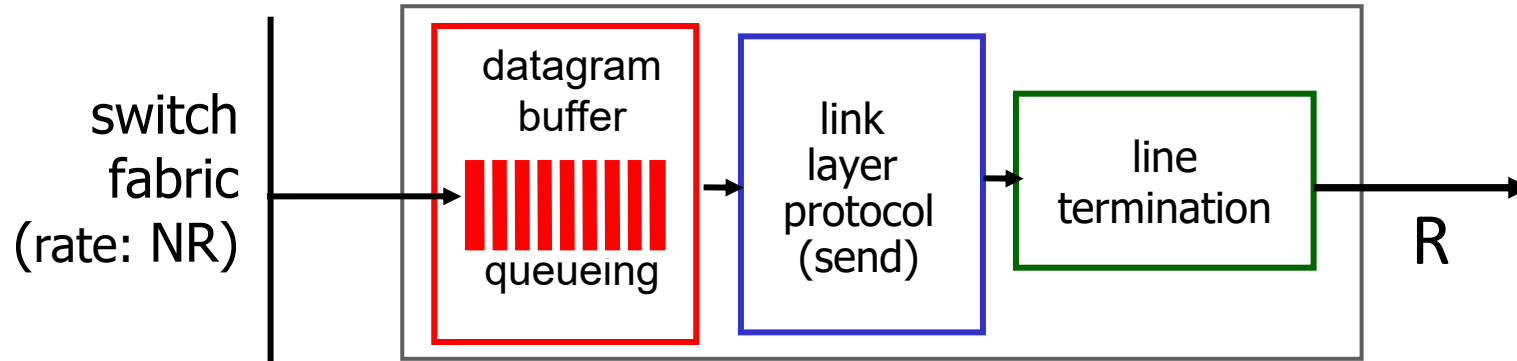
**output port contention:** only one red datagram can be transferred. lower red packet is *blocked*



one packet time later: green packet experiences HOL blocking



# Output port queuing



This is a really important slide

- **Buffering** required when datagrams arrive from fabric faster than link transmission rate. **Drop policy:** which datagrams to drop if no free buffers?



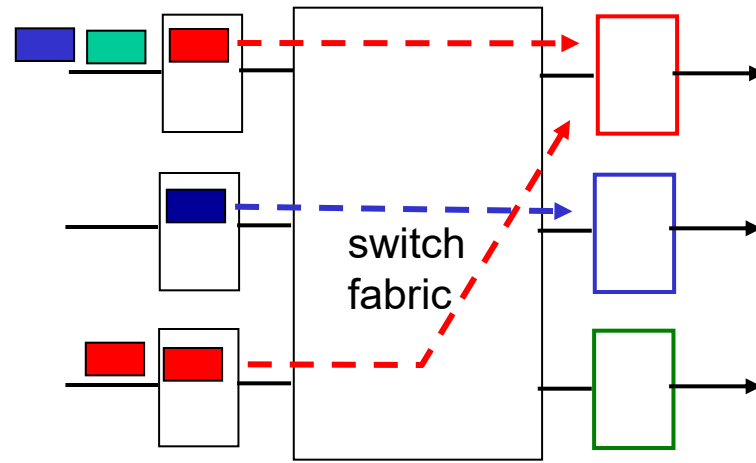
Datagrams can be lost due to congestion, lack of buffers

- **Scheduling discipline** chooses which datagram among queued datagrams for transmission

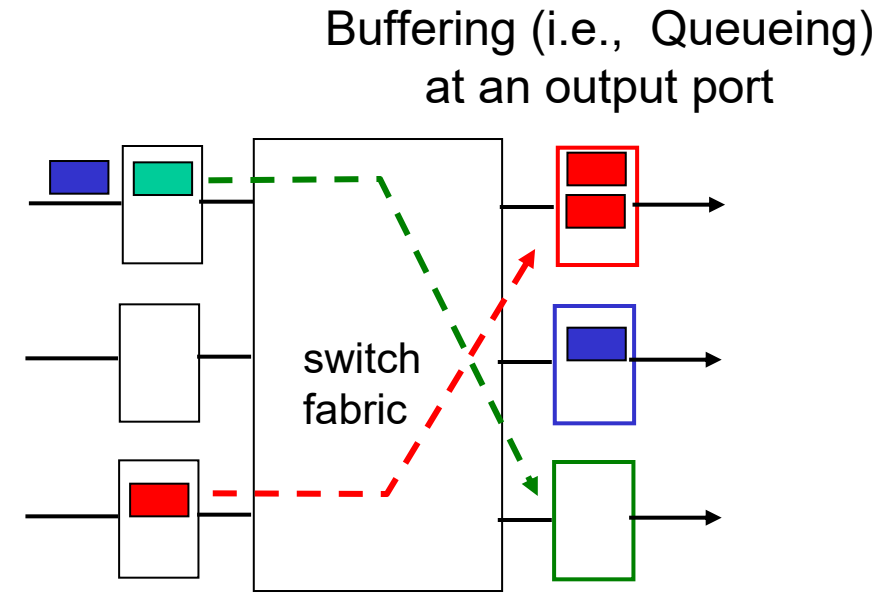


Priority scheduling – who gets best performance, network neutrality

# Output port queuing



at  $t$ , more than one packet  
moves from input to output



one packet time later

- **buffering** when arrival rate via switch exceeds output line speed
- *queueing (delay) and loss due to output port buffer overflow!*

# How much buffering?

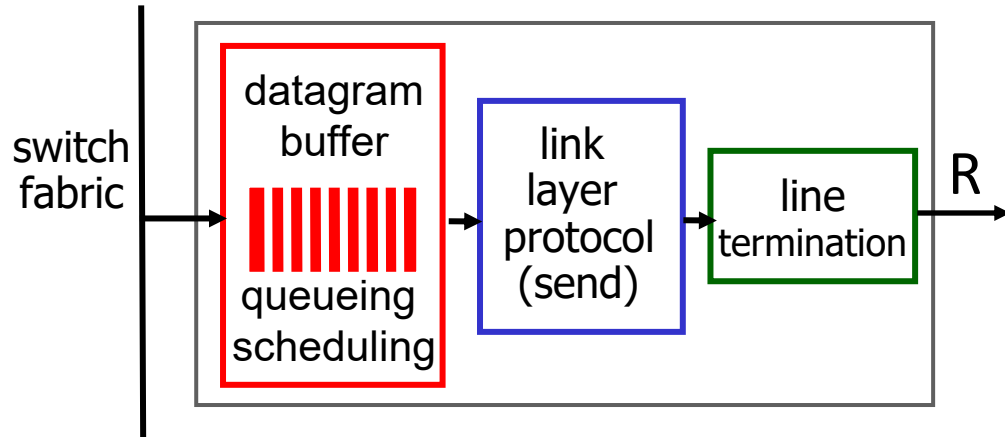
- RFC 3439 rule of thumb: average buffering equal to “typical” RTT (say 250 msec) times link capacity  $C$ 
  - e.g.,  $C = 10$  Gbps link: 2.5 Gbit buffer

- more recent recommendation: with  $N$  flows, buffering equal to

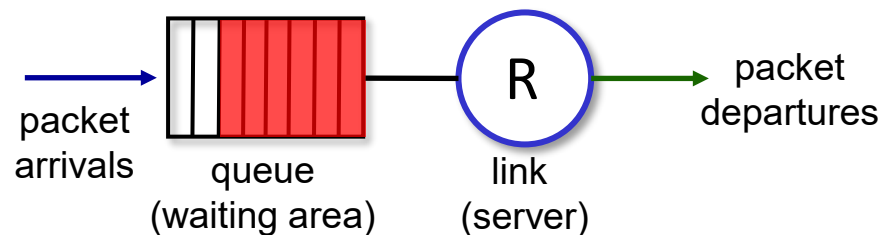
$$\frac{RTT \cdot C}{\sqrt{N}}$$

- but *too* much buffering can increase delays (particularly in home routers)
  - long RTTs: poor performance for realtime apps, sluggish TCP response
  - recall delay-based congestion control: “keep bottleneck link just full enough (busy) but no fuller”

# Buffer Management



## Abstraction: queue



## buffer management:

- **drop:** which packet to add, drop when buffers are full
  - **tail drop:** drop arriving packet
  - **priority:** drop/remove on priority basis
- **marking:** which packets to mark to signal congestion (ECN, RED)
  - ECN: Explicit Congestion Notification
  - RED: Random Early Detection

# Packet Scheduling: FCFS

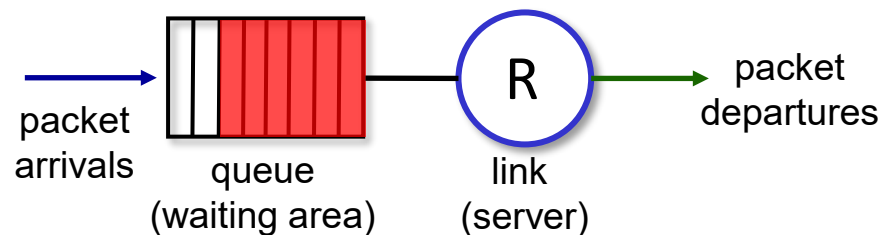
**packet scheduling:** deciding which packet to send next on link

- first come, first served
- priority
- round robin
- weighted fair queueing

**FCFS:** packets transmitted in order of arrival to output port

- also known as: First-in-first-out (FIFO)
- real world examples?

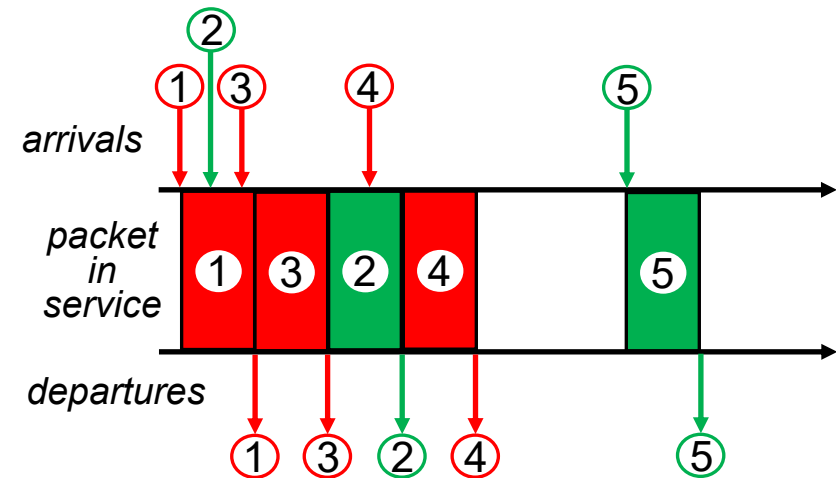
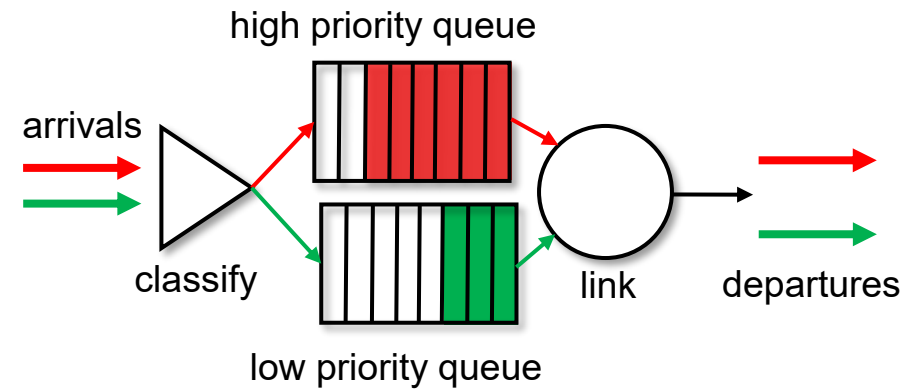
Abstraction: queue



# Scheduling policies: priority

## *Priority scheduling:*

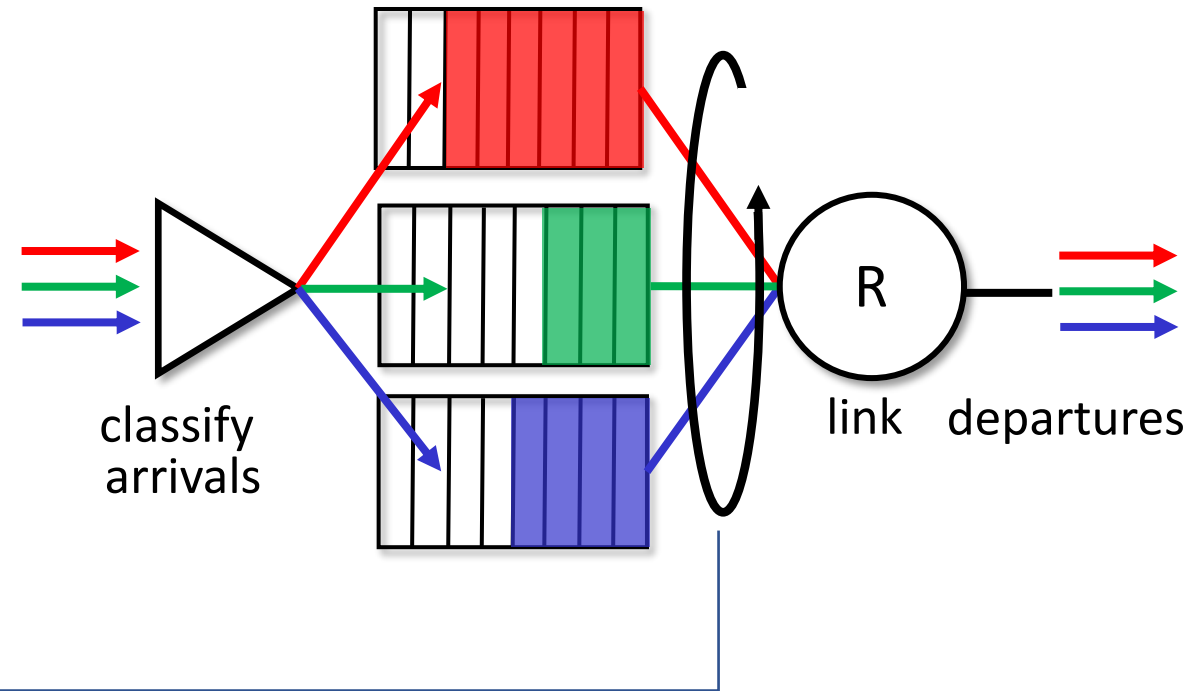
- arriving traffic classified, queued by class
  - any header fields can be used for classification
- send packet from highest priority queue that has buffered packets
  - FCFS within priority class



# Scheduling policies: round robin

## *Round Robin (RR) scheduling:*

- arriving traffic classified, queued by class
  - any header fields can be used for classification
- server cyclically, repeatedly scans class queues, sending one complete packet from each class (if available) in turn



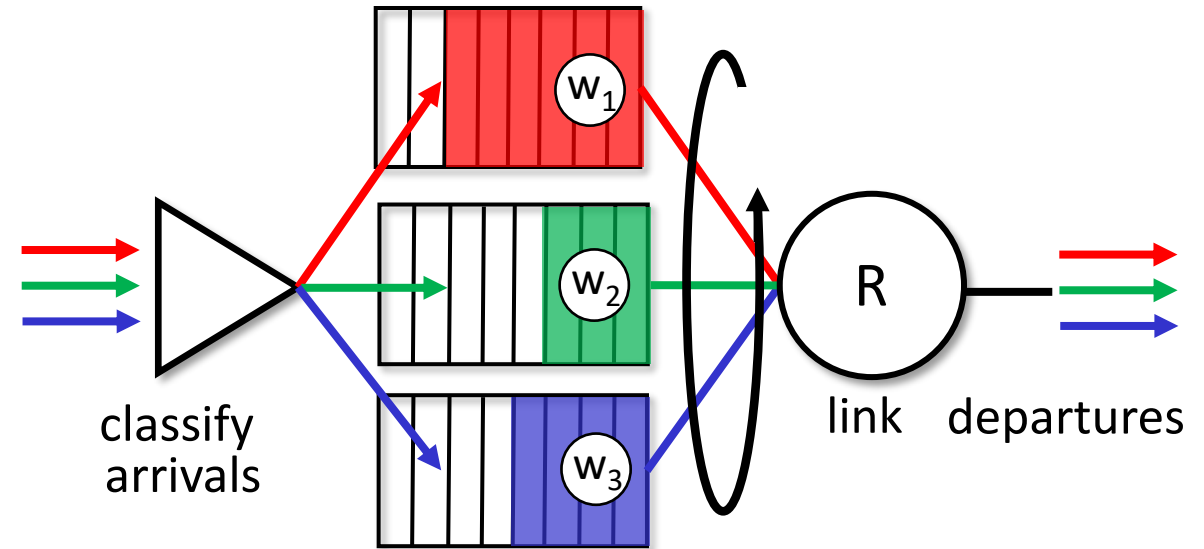
# Scheduling policies: weighted fair queueing

## *Weighted Fair Queuing (WFQ):*

- generalized Round Robin
- each class,  $i$ , has weight,  $w_i$ , and gets weighted amount of service in each cycle:

$$\frac{w_i}{\sum_j w_j}$$

- minimum bandwidth guarantee (per-traffic-class)





# Sidebar: Network Neutrality

What is network neutrality?

- *definition*: All data on the Internet is treated equally by Internet Service Providers (ISPs) and governments, regardless of content, user, platform, application, or device.
- *technical*: how an ISP should share/allocation its resources
  - packet scheduling and buffer management are the *mechanisms*.
- *social, economic* principles
  - protecting free speech
  - encouraging innovation, competition
- enforced *legal* rules and policies

*Different countries have different “takes” on network neutrality*

# Sidebar: Network Neutrality

2015 US FCC *Order on Protecting and Promoting an Open Internet*: three “clear, bright line” rules:

- **no blocking** ... “shall not block lawful content, applications, services, or non-harmful devices, subject to reasonable network management.”
- **no throttling** ... “shall not impair or degrade lawful Internet traffic on the basis of Internet content, application, or service, or use of a non-harmful device, subject to reasonable network management.”
- **no paid prioritization** ... “shall not engage in **paid prioritization**”
  - **Paid prioritization:** the management of an ISP’s network to directly or indirectly favor some traffic over other traffic, including through the use of techniques such as traffic shaping, prioritization, resource reservation, or other forms of preferential traffic management, either.

# ISP: telecommunications or information service?

Is an ISP a “telecommunications service” or an “information service” provider?

- the answer *really* matters from a regulatory standpoint!

US Telecommunication Act of 1934 and 1996:

- *Title II*: imposes “common carrier duties” on *telecommunications services*: reasonable rates, non-discrimination and *requires regulation*
  - *Title I*: applies to *information services*:
    - no common carrier duties (*not regulated*)
    - but grants FCC authority “... as may be necessary in the execution of its functions”.
- ✓ Federal Communications Commission (FCC): <https://www.fcc.gov/>